

Algorithms and Data Structures I.

Practice 3

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

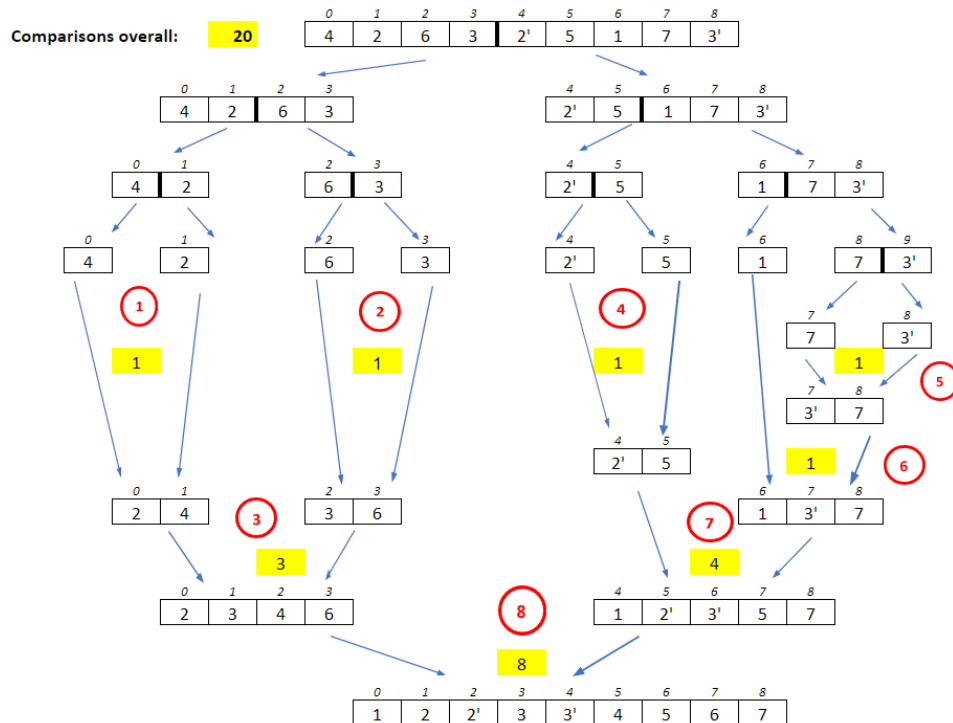
April 18, 2024

Content in this Practice I

- ① Review last week
 - Merge Sort - Representation example
- ② Divide and Conquer algorithms
 - Quicksort
 - Quicksort - Example
 - Quicksort - Step by step
 - Quicksort - Structogram
 - Quicksort - Summary
 - Quicksort - Another detailed example
- ③ Data Structure - Stack
 - Stack

Review last week

Merge Sort - Representation example



Merge Sort - Representation example

1

START

	0	1	2	3	4	5	6	7	8
A	4	2	6	3	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

2

START

	0	1	2	3	4	5	6	7	8
A	2	4	6	3	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

3

START

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

4

START

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

merge(B,A,0,1,2)

(from B to A)

END

	0	1	2	3	4	5	6	7	8
A	2	4	6	3	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

comparisons 1

merge(B,A,2,3,4)

(from B to A)

END

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	4	2	6	3	2'	5	1	7	3'

1

merge(A,B,0,2,4)

(from A to B)

END

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

3

merge(B,A,4,5,6)

(from B to A)

END

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

1

Merge Sort - Representation example

5

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	7	3'

START

6

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	3'	7

START

7

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	2'	5	1	3'	7

START

8

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	1	2'	3'	5	7

merge(A,B,7,8,9)

(from A to B)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	7	3'
B	2	3	4	6	2'	5	1	3'	7

1

END

merge(B,A,6,7,9)

(from B to A)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	2'	5	1	3'	7

1

END

merge(A,B,4,6,9)

(from A to B)

	0	1	2	3	4	5	6	7	8
A	2	4	3	6	2'	5	1	3'	7
B	2	3	4	6	1	2'	3'	5	7

4

END

merge(B,A,0,4,9)

(from B to A)

	0	1	2	3	4	5	6	7	8
A	1	2	2'	3	3'	4	5	6	7
B	2	3	4	6	1	2'	3'	5	7

8

Divide and Conquer algorithms

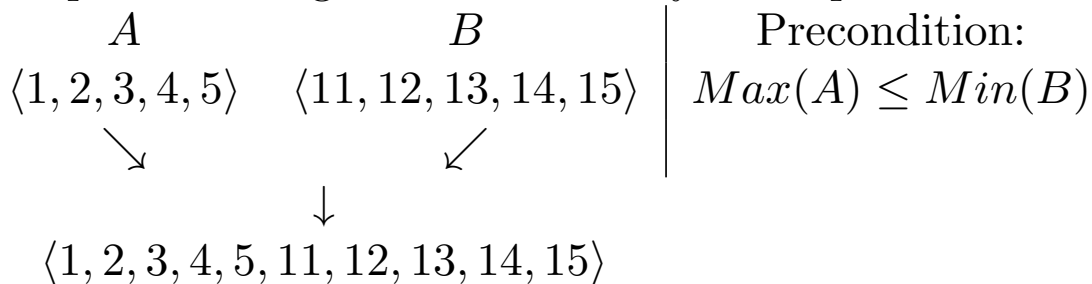
Quicksort

- QuickSort: *a sorting algorithm based on the Divide and Conquer algorithm that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.*
- QuickSort:
 - Choose a pivot element, place the pivot at its correct position in the sorted array, and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.
 - Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.

Quicksort - Main idea

- Main Idea:

If the left and the right arrays are sorted and every element in the left array is smaller than any element in the right, I can put them together without any extra process.



Quicksort - Example

Array:	$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$
Choosing the <i>PIVOT</i> item:	$\langle 12, 31, 23, 20, \underline{14}, 55, 11 \rangle$
Swap with the last element:	$\langle 12, 31, 23, 20, 11, 55, \underline{14} \rangle$
Look for $A[i] > \textit{PIVOT}$:	$\langle 12, \underline{31}, 23, 20, 11, 55, \underline{14} \rangle$
Set $j := i + 1$:	$\langle 12, \underline{31}, \underline{23}, 20, 11, 55, \underline{14} \rangle$
Look for $A[j] < \textit{PIVOT}$:	$\langle 12, \underline{31}, 23, 20, \underline{11}, 55, \underline{14} \rangle$
SWAP $\underline{A[i]}$ and $\underline{A[j]}$:	$\langle 12, \underline{11}, 23, 20, \underline{31}, 55, \underline{14} \rangle$
Set $i := i + 1$:	$\langle 12, 11, \underline{23}, 20, \underline{31}, 55, \underline{14} \rangle$
Continue $A[j] < \textit{PIVOT}$:	$\langle 12, 11, \underline{23}, 20, \underline{31}, 55, \underline{14} \rangle$
j reaches the <i>PIVOT</i> :	$\langle 12, 11, \underline{23}, 20, 31, 55, \underline{14} \rangle$
SWAP $\underline{A[i]}$ with the <i>PIVOT</i> :	$\langle 12, 11, \underline{14}, 20, 31, 55, \underline{23} \rangle$

$\langle A \text{ (small)} \rangle \textit{PIVOT} \langle B \text{ (big)} \rangle :$ | $\langle 12, 11 \rangle \underline{14} \langle 20, 31, 55, 23 \rangle$

Quicksort - Initialize PIVOT

Array:	$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$
Choosing the <i>PIVOT</i> item:	$\langle 12, 31, 23, 20, \textcolor{brown}{14}, 55, 11 \rangle$
Swap with the last element:	$\langle 12, 31, 23, 20, 11, 55, \textcolor{brown}{14} \rangle$

$\text{partition}(A : \mathcal{T}[] ; p, r : \mathbb{N}) : \mathbb{N}$

$i := \text{random}(p, r)$ // Select a random pivot

$\text{swap}(A[i], A[r])$ // $A[r]$ is the pivot

$i := p$

Quicksort - Looking for larger elements

Look for $A[i] > \textit{PIVOT}$:	$\langle 12, 31, 23, 20, 11, 55, 14 \rangle$
$i := p$	
$i < r \wedge A[i] \leq A[r]$	
	$i := i + 1$

Quicksort - Looking for smaller elements and swap

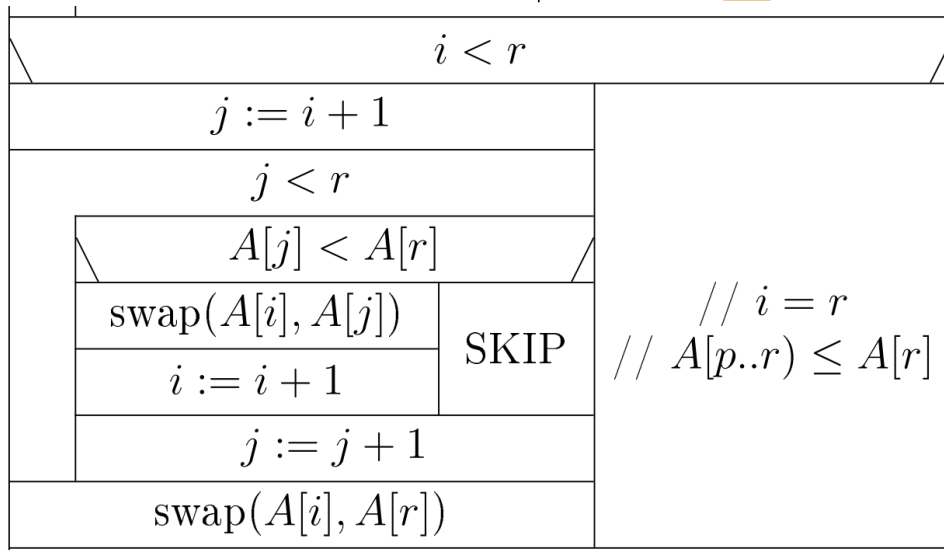
Set $j := i + 1$:	$\langle 12, \textcolor{blue}{31}, \textcolor{red}{23}, 20, 11, 55, \textcolor{brown}{14} \rangle$
Look for $A[j] < \textcolor{brown}{PIVOT}$:	$\langle 12, \textcolor{blue}{31}, 23, 20, \textcolor{red}{11}, 55, \textcolor{brown}{14} \rangle$
SWAP $A[i]$ and $A[j]$:	$\langle 12, \textcolor{blue}{11}, 23, 20, \textcolor{blue}{31}, 55, \textcolor{brown}{14} \rangle$
Set $i := i + 1$:	$\langle 12, 11, \textcolor{blue}{23}, 20, \textcolor{red}{31}, 55, \textcolor{brown}{14} \rangle$
Continue $A[j] < \textcolor{brown}{PIVOT}$:	$\langle 12, 11, \textcolor{blue}{23}, 20, \textcolor{red}{31}, 55, \textcolor{brown}{14} \rangle$

$i < r$	
$j := i + 1$	$// i = r$ $// A[p..r) \leq A[r]$
$j < r$	
$A[j] < A[r]$	
swap($A[i], A[j]$)	
$i := i + 1$	
SKIP	

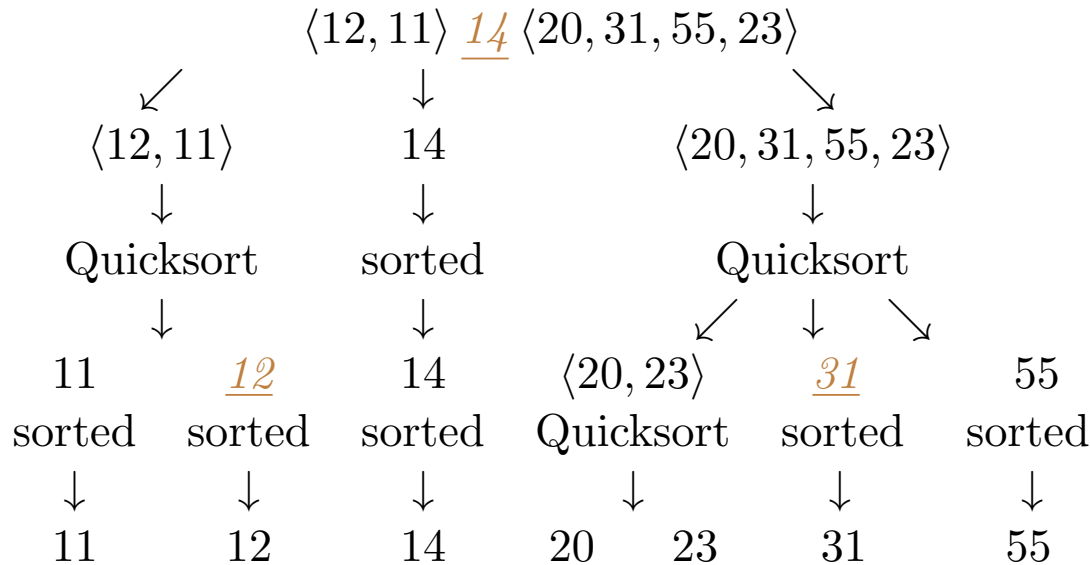
Quicksort - Final

j reaches the <i>PIVOT</i> :	$\langle 12, 11, \mathbf{23}, 20, 31, 55, \mathbf{14} \rangle$
SWAP $A[\mathbf{i}]$ with the <i>PIVOT</i> :	$\langle 12, 11, \mathbf{14}, 20, 31, 55, \mathbf{23} \rangle$

$\langle \text{sorted} \rangle$ *PIVOT* $\langle \text{sorted} \rangle$: | $\langle 12, 11 \rangle$ 14 $\langle 20, 31, 55, 23 \rangle$

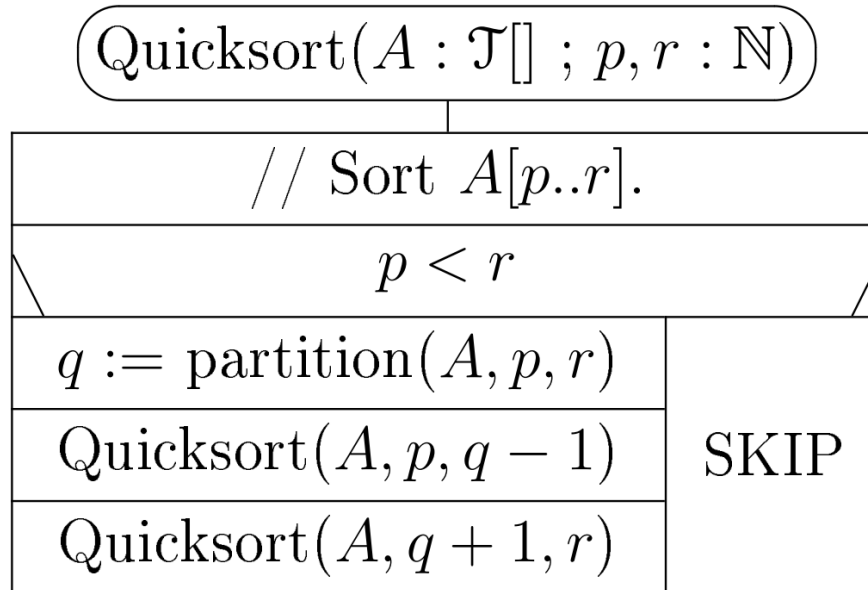


Quicksort - Recursion

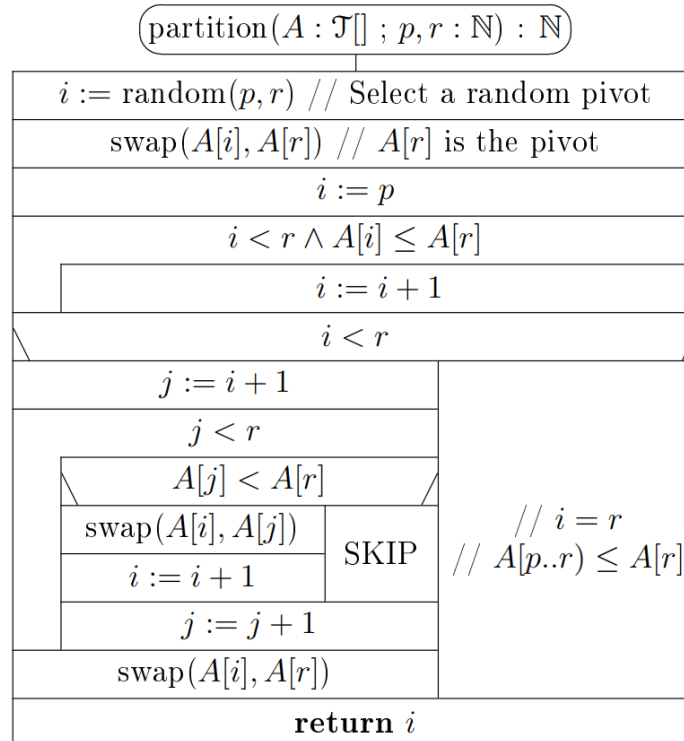


sorted: $\langle 11, 12, 14, 20, 23, 31, 55 \rangle$

Quicksort - Recursion Structogram



Quicksort - Partition Structogram



Quicksort - Summary

Time complexity:

- $mT(n), AT(n) \in O(n * \log(n))$
- $MT(n) \in O(n^2)$: Worst-case scenario with a unbelievably low possibility.

Space complexity

- $O(1)$
- "In-place" algorithm - it sorts the elements within the array with at most a constant amount of variables outside the array at any given time.

Not Stable: Quicksort can be stable but it typically isn't implemented that way. Making it stable either requires order N storage (as in a naive implementation) or a bit of extra logic for an in-place version.

Quicksort - Another example - First partitioning

1	2	3	4	5	6	7	8	9	10
8	12	5	14	10	2	9	17	3	9

Partition(A,1,10)

x=9

1	i	j	4	5	6	7	8	9	10
8	12	5	14	10	2	9	17	3	

Csere(A[i],A[j])

1	2	i	4	5	j	7	8	9	10
8	5	12	14	10	2	9	17	3	

Csere(A[i],A[j])

1	2	3	i	5	6	7	8	j	10
8	5	2	14	10	12	9	17	3	

Csere(A[i],A[j])

1	2	3	4	i	6	7	8	9	j
8	5	2	3	10	12	9	17	14	

tengely a helyére

1	2	3	4	i	6	7	8	9	10
8	5	2	3	9	12	9	17	14	10

Első partícionálás vége

- We pick the last element as the pivot, and leave an 'empty slot' there.
- i starts at the beginning and stops at the element 12, because that's the first larger element. j starts from the element after i, and stops right there because 5 < x. We swap (A[2],A[3]), and then i=3, j=4
- j skips every element that is larger or equal, and stops at 2, the first smaller element. Swap(A[3],A[6]), i=4, j=7
- j until the 9th element, Swap(A[4],A[9]), i=5, j=10
- j reaches the 'empty slot' and we swap it with the element at i
- Return the value of i

Quicksort - Subsequent runs

1	2	3	4	5	6	7	8	9	10
8	5	2	3	9	12	9	17	14	10

partition(A,1,10)=5

1	2	3	4	5	6	7	8	9	10
8	5	2	3	9	12	9	17	14	10

x=3

1	i	3	4	5	6	7	8	9	10
2	3	8	5	9	12	9	17	14	10

Következik: partititon(A,1,4)

tengely:A[4]

partition(A,1,4)=2

1	i	3	4	5	6	7	8	9	10
2	3	8	5	9	12	9	17	14	10

x=5

1	2	i	4	5	6	7	8	9	10
2	3	5	8	9	12	9	17	14	10

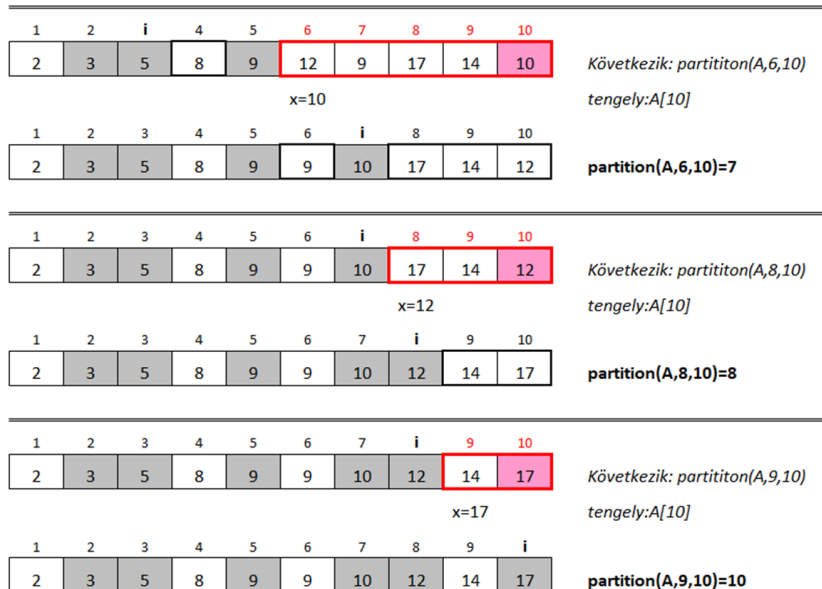
Következik: partititon(A,3,4)

tengely:A[4]

partition(A,3,4)=3

- partition(A,1,10)
8,5,2,3,+9,12,9,17,14,10
Comparisons:
9
- partition(A,1,4)
2,+3,8,5
Comparisons:
3 (Sum=12)
- partition(A,3,4)
+5,8
Comparisons:
1 (Sum=13)

Quicksort - Subsequent runs



- $\text{partition}(A, 6, 10)$:
9, +10, 17, 14, 12
comparisons:
4 (Sum=17)
- $\text{partition}(A, 8, 10)$:
+12, 14, 17
comparisons:
2 (Sum=19)
- Partition(9, 10)
14, +17
comparisons:
1 (Sum=20)

Data Structure - Stack

Stack

Illustration problem: You just moved to a new room with all your stuff still in boxes. You have limited space, how do you save some space?

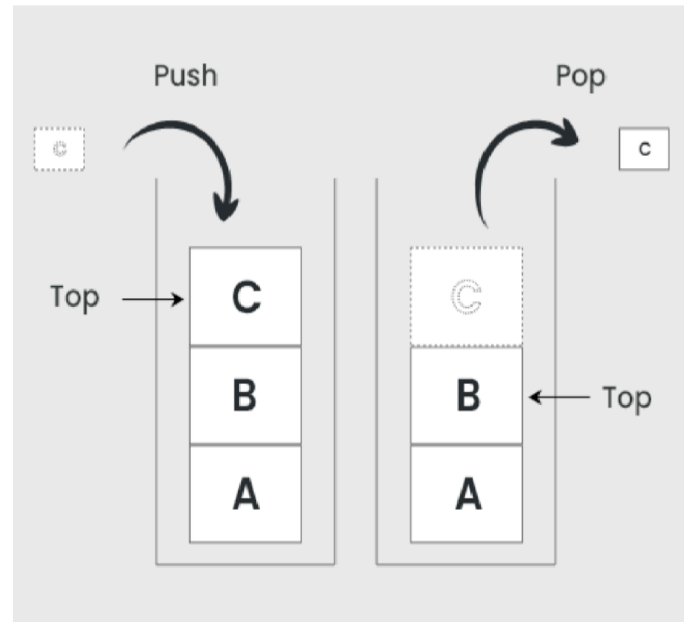
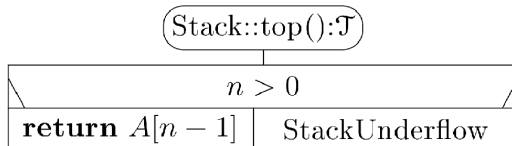
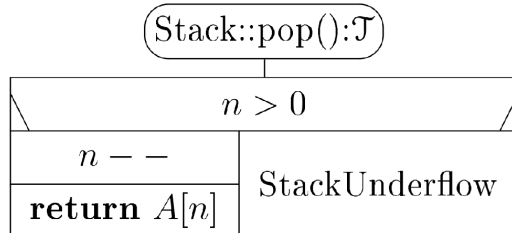
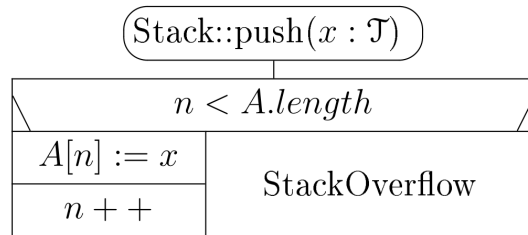
You put the boxes on top of each other.

- Bring in the new box into the room.
- Put it on top of the previous box.

Stack

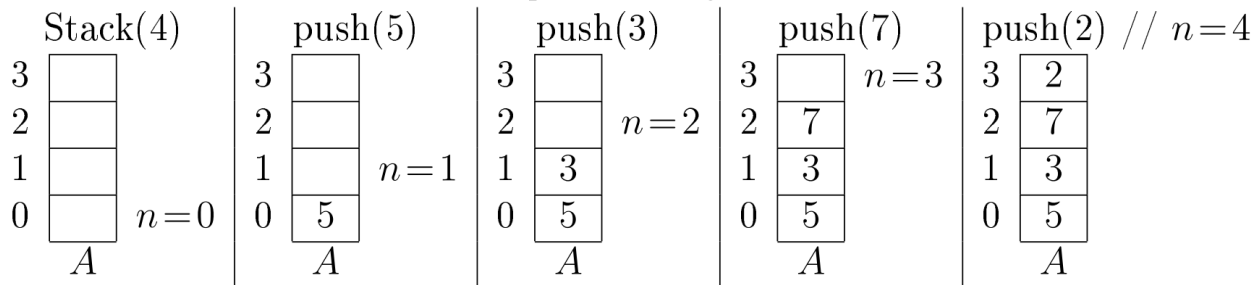
Stack
$-A : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the max. size of the stack
$-n : \mathbb{N}$ // $n \in 0..A.length$ is the actual size of the stack
$+ \text{Stack}(m : \mathbb{N}) \{A := \mathbf{new} \mathcal{T}[m] ; n := 0\}$ // create an empty stack
$+ \text{push}(x : \mathcal{T})$ // push x onto the top of the stack
$+ \text{pop}() : \mathcal{T}$ // remove and return the top element of the stack
$+ \text{top}() : \mathcal{T}$ // return the top element of the stack
$+ \text{isFull}() : \mathbb{B} \{\mathbf{return} \ n = A.length\}$
$+ \text{isEmpty}() : \mathbb{B} \{\mathbf{return} \ n = 0\}$
$+ \text{setEmpty}() \{n := 0\}$ // reinitialize the stack
$+ \sim \text{Stack}() \{ \mathbf{delete} \ A \}$

Stack



Stack

Some operations of a Stack



Ending

Thank you for your attention!